

Reviewer Pack — Hypercontractive Term-Pair Contraction Defect Bounds Monoton...

Entry 684e59090c85 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 13:47:31 UTC

Hypercontractive Term-Pair Contraction Defect Bounds Monotone CLIQUE DNF

Entry ID: 684e59090c85 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 13:47:31 UTC

1. Conjecture

Field A (mathematical branch): Fourier-analytic hypercontractivity — Bonami-Beckner noise kernel applied SYNTACTICALLY to the symmetric-difference distribution of TERM-SUPPORT pairs of a monotone DNF, in the Kahn-Kalai-Linial / Friedgut / Bonami / Mossel tradition. The invariant is a Rényi-2-to-Rényi-4-style log-ratio of the noise-mass $D_\rho(F) := (1/s^2) \cdot \sum_{i,j} \rho^{|T_i \Delta T_j|}$ at two scales $\rho \in \{1/2, 1/4\}$ — a non-ring functional (uses $|\cdot|$ on integer-valued symmetric differences and log-ratio of exponential sums, with no polynomial F_q extension since ρ^t is real and the symdiff cardinality is not preserved under ring lifts), so the bridge is Aaronson-Wigderson algebrization-safe. Distinct from blacklisted Bonami 4/2 ratio of the centered CLAUSE-INDICATOR polynomial Q_F on DPLL (that was a truth-table-flavored single polynomial, not a term-pair kernel) and from Hypercontractive Term-Support Stable Rank $\sigma(K) = (\sum \lambda)^2 / \sum \lambda^2$ (Gram-spectrum invariant via eigendecomposition, which crashed in dense matrix code; here we never form eigenvalues — only the scalar log-ratio of two noise-kernel sums). An arXiv search ‘Bonami-Beckner’ AND (‘monotone DNF’ OR ‘CLIQUE indicator’) applied as a TERM-PAIR symdiff log-ratio returns 0 direct hits with <5 adjacent papers (all on truth-table-side hypercontractivity, never on the DNF’s term family). **Field B** (complexity object): Monotone DNF leaf-size $L(F) := \sum_i |T_i|$ for the k -CLIQUE indicator on K_v with $k = 3$ fixed and $n = v(v-1)/2$ edge variables, in the Razborov 1985 / Andreev / Karchmer-Wigderson MONOTONE CLIQUE regime. The invariant is SYNTACTIC on the DNF (two DNFs with the same Boolean function can have very different η), shielding it from the Razborov-Rudich natural-proofs barrier.

Statement:

For monotone DNF $F = \bigvee_{i=1}^s T_i$ on n variables ($T_i \subseteq [n]$, $s \geq 2$), define the Bonami noise mass $D_\rho(F) := (1/s^2) \cdot \sum_{i,j=1}^s \rho^{|T_i \Delta T_j|}$ for $\rho \in (0,1)$, and the hypercontractive contraction defect $\eta(F) := \log_2 D_{1/2}(F) - 2 \cdot \log_2 D_{1/4}(F)$. Conjecture: there exist absolute constants $C_1 \leq 6$ and $c_2 \geq 0.10$ such that (i) for every monotone DNF F with leaf-size L , $\eta(F) \leq C_1 \cdot \log_2(L + 2)$, and (ii) for the canonical minimal 3-CLIQUE DNF F^*_v on K_v (one term per triangle, $|T_i| = 3$), $\eta(F^*_v) \geq c_2 \cdot \sqrt{v}$ for all $v \geq 5$. A single F violating (i) with margin > 0.5 or a single $v \geq 5$ with $\eta(F^*_v) < c_2 \cdot \sqrt{v} - 0.05$ refutes the conjecture.

Rationale (proposer’s reasoning):

D_ρ measures how clustered the term family is in Hamming geometry; the 1/2-vs-1/4 log-ratio $\eta(F)$ is a discrete analogue of a Bonami 2→4 hypercontractive contraction taken on

the term-pair empirical measure, isolating the ‘shape’ of the symmetric-difference distribution beyond its mean. For poly-leaf-size DNFs the term family is small/sparse, forcing the symdiff distribution to concentrate at large values and bounding η by $O(\log L)$; for the 3-CLIQUE family the $C(v,3)$ triangles share a rigid combinatorial overlap pattern (edge-sharing pairs contribute symdiff 4, otherwise 6), creating a two-mode mixture whose mass at the small-symdiff mode scales like $1/v$, producing a predictable $\sqrt{v}$ growth in the 2/4 contraction ratio. Because η depends on the multiset of term-supports (not the truth table), it is shielded from Razborov–Rudich, and because it uses only $|\cdot|$ and exponential sums it does not algebrize.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b83c5653ead8d932

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Conjecture supported if (i) $\max \eta/\ln(L+2) \leq 6$ and (ii) $\min \eta/\sqrt{v} \geq 0.10$ across 30 seeds per family; falsified otherwise.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL PROOFS	SAFE	0.85	SAFE	SAFE
ALGEBRIZATION	SAFE	0.86	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_random_dnf(n, k, s):
    terms = [set(random.sample(range(1, n+1), k)) for _ in range(s)]
    return terms

def generate_adversarial_dnf(n, k, s):
    base_terms = set(random.sample(range(1, n+1), k))
    adversarial_terms = [base_terms.union({i}) for i in range(1, n+1) if i not in base_terms]
```

```

terms = random.sample(adversarial_terms, s)
return terms

def generate_clique_dnf(n, v):
    edges = list(combinations(range(1, n+1), 2))
    clique_terms = []
    for triangle in combinations(edges, 3):
        term = set()
        for edge in triangle:
            term.update(edge)
        clique_terms.append(term)
    return clique_terms[:v]

def symdiff_count(A, B):
    return sum(1 for x in A if x not in B) + sum(1 for y in B if y not in A)

def compute_D_rho(F, rho):
    count = 0
    n = len(F[0])
    for i in range(n):
        for j in range(i+1, n):
            diff_count = symdiff_count(F[i], F[j])
            count += rho ** diff_count
    return count / (len(F) ** 2)

def compute_eta(F):
    D_half = compute_D_rho(F, 0.5)
    D_quarter = compute_D_rho(F, 0.25)
    return math.log2(D_half) - 2 * math.log2(D_quarter)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [12, 18, 24, 30, 36]
    k_values = [2, 3, 4]
    s_values = [n * n, 2 * n * n, n * n * n] #  $n^{1.5}$  is  $n^3$  in this context

    results = []

    for n in n_values:
        for k in k_values:
            for s in s_values:
                F = generate_random_dnf(n, k, s)
                eta = compute_eta(F)
                results.append((eta, math.log2(s * k + 2)))

                if len(results) >= 30: # Ensure at least 30 instances per seed
                    break
            else:
                continue
        else:
            break
    else:
        continue
    break

max_ratio = max(eta / log_L for eta, log_L in results)

```

```

conjecture_holds = max_ratio <= 6

return {
    "metric_name": "max_eta_over_logL",
    "metric_value": max_ratio,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]]
    if not seeds:
        seeds = [2*i + 3 for i in range(5, 8)] # Default to first 30 primes

    results = []

    for seed in seeds:
        trial = run_trial(seed)
        print(f"TRIAL: {trial}")
        results.append(trial)

    max_ratio = max(result["metric_value"] for result in results if result["conjecture_holds"])
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={max_ratio} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in enumerate(results, start=1) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"max_eta_over_logL\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=budget_exceeded n_tested=<k>")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7d2086c.py", line 98, in <module>
    trial = run_trial(seed)
            ~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f7d2086c.py", line 59, in run_trial
    s_values = [n * n, 2 * n * n, n * n * n] # n^{1.5} is n^3 in this context
                ^

```

UnboundLocalError: cannot access local variable 'n' where it is not associated with a value

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to unbound local variable 'n' before producing results | next: Debug test code to resolve UnboundLocalError and ensure proper variable initialization

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	383563
2	propose	claude_max	opus	0	0	337197
3	preregistration	ollama_remote	qwen3:8b	0	0	273731
4	novelty	ollama_remote	qwen3:8b	0	0	168411
5	novelty	ollama_remote	qwen3:8b	0	0	199079
6	test_gen	mistral	codestral-latest	0	0	317241
7	test_gen	mistral	codestral-latest	0	0	318697
8	test_gen	mistral	codestral-latest	0	0	314277
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	268484
10	judge	ollama_remote	qwen3:8b	0	0	131413

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 2712094 ms total latency. Provider mix: {'claude_max': 2, 'ollama_remote': 5, 'mistral': 3}

(full prompt+response transcripts available in research/audit/684e59090c85.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/684e59090c85.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/684e59090c85.tar.gz (if generated)